

PROTO2

Connected Management Hub

Documentation du projet

Plateforme	Raspberry Pi 5
Outil principal	Node-RED
Services connectés	Minecraft · Discord · Dashboard web
Plugins utilisés	node-red-contrib-discord-advanced · @tomsith/node-red-contrib-minecraft · @flowfuse/node-red-dashboard
Dépôt GitHub	https://github.com/mdc0106/PROTO2

Vue d'ensemble du projet

PROTO2 est un hub de gestion connecté qui relie trois services ensemble : un serveur Minecraft, un bot Discord et un tableau de bord web. Tout fonctionne sur un Raspberry Pi 5 via Node-RED.

L'objectif est de pouvoir gérer le serveur Minecraft directement depuis Discord (en tapant des commandes) ou depuis le tableau de bord web, sans avoir à toucher au serveur manuellement.

Comment les services communiquent

Les trois services ne se parlent pas directement. Node-RED est le centre qui reçoit les messages de chaque service et envoie les bonnes instructions aux autres.

Discord

Un utilisateur tape une commande dans le canal Discord. Le bot lit le message et l'envoie à Node-RED. Node-RED analyse la commande, contacte le serveur Minecraft via RCON, et renvoie la réponse dans Discord.

Dashboard web

Le tableau de bord est accessible à l'adresse `cabma201:1880/dashboard`. Il affiche en temps réel les joueurs connectés, l'état du serveur, et les statistiques matérielles du Raspberry Pi. On peut aussi kicker ou bannir un joueur depuis cette interface.

Minecraft (RCON)

RCON est un protocole qui permet d'envoyer des commandes à un serveur Minecraft à distance, comme si on les tapait dans la console du serveur. Node-RED s'y connecte sur le port 25575.

Les 5 flux Node-RED

Le projet est organisé en 5 onglets (flux) dans Node-RED. Chaque flux a un rôle précis. Ils fonctionnent tous en parallèle, de façon indépendante.

Flux 1 — Bot Setup (Configuration du bot)

Ce flux configure le bot Discord au démarrage de Node-RED. Il définit le statut du bot (le texte affiché sous son nom) et confirme dans le terminal que tout est prêt.

1**Inject (démarrage)**

Se déclenche 1 seconde après le démarrage de Node-RED. Envoie un signal de départ.

2**bot-status-func**

Prépare le message de statut : "Surveillance PROTO2 | !help". Ce texte apparaîtra sous le nom du bot dans Discord.

3**discordActivity**

Applique le statut au bot via l'API Discord.

4**bot-ready-debug**

Affiche "Bot PROTO2 actif" dans le panneau debug de Node-RED pour confirmer que tout fonctionne.

Flux 2 — Discord Commands (Commandes Discord)

Ce flux écoute les messages envoyés dans le canal Discord et les traduit en commandes Minecraft ou en actions. C'est le flux le plus important.

Étape 1 — Réception et tri

1**cmd-listen (discordMessage)**

Écoute en permanence les messages du canal Bot Tester. Dès qu'un message arrive, il est envoyé au nœud suivant.

2**cmd-router (function)**

Lit le message et décide où l'envoyer selon la commande détectée. Il a 8 sorties, une par commande.

Commandes reconnues par le routeur :

!survival	Met tous les joueurs en mode Survie
!creative	Met tous les joueurs en mode Créatif
!day	Passe l'heure du serveur à jour
!night	Passe l'heure du serveur à nuit
!save	Sauvegarde le monde manuellement
!list	Affiche la liste des joueurs connectés
!kick <joueur>	Expulse un joueur du serveur
!ban <joueur>	Bannit un joueur du serveur

Étape 2 — Exécution

Chaque sortie du routeur est connectée à un nœud RCON qui envoie la commande correspondante au serveur Minecraft. Le serveur exécute la commande et renvoie une réponse.

Étape 3 — Réponse Discord

La réponse du serveur est mise en forme par une fonction et envoyée dans le canal Discord par le nœud discordMessageManager.

Système de ticket (!report)

La commande !report fonctionne différemment des autres. Elle n'envoie pas de commande Minecraft, mais crée plutôt un ticket de support.

1**cmd-report-listen**

Écoute les messages qui commencent par !report dans le canal Discord.

2**cmd-report-filter**

Vérifie que le message est bien un !report valide et extrait le texte du signalement.

3**cmd-report-ack**

Envoie un message de confirmation dans Discord : le ticket a bien été reçu.

4**cmd-report-api**

Appelle l'API Discord pour créer automatiquement un nouveau canal nommé ticket-[auteur]-[timestamp].

5**cmd-report-dashboard**

Ajoute le ticket au tableau de la page Controls du dashboard web.

Flux 3 — Server Monitor (Surveillance du serveur)

Ce flux interroge le serveur Minecraft toutes les 30 secondes et affiche un résumé dans Discord : combien de joueurs sont connectés, la version, et si le serveur est en ligne.

1

mon-inject

Minuterie : se déclenche toutes les 30 secondes.

2

mon-serverinfo

Interroge le serveur Minecraft pour obtenir ses informations (joueurs, version, état).

3

mon-status-func

Met en forme les informations : construit le message à envoyer dans Discord avec les données reçues.

4

mon-discord-send

Envoie le message de statut dans le canal Discord.

Flux 4 — Player Events (Événements joueurs)

Ce flux détecte en temps réel quand un joueur rejoint ou quitte le serveur, et poste un message dans Discord.

1

evt-inject

Minuterie : se déclenche toutes les 10 secondes.

2

evt-serverinfo

Récupère la liste actuelle des joueurs connectés.

3

evt-diff-func

Compare la liste actuelle avec la liste d'il y a 10 secondes. Si un nom est nouveau → il a rejoint. Si un nom a disparu → il est parti.

4

evt-discord-notify

Envoie un message dans Discord pour chaque événement détecté.

Exemples de messages envoyés :

-  Steve a rejoint le serveur !
-  Alex a quitté le serveur.

Flux 5 — Dashboard (Tableau de bord)

Ce flux alimente le tableau de bord web accessible à l'adresse `cabma201:1880/dashboard`. Il met à jour les différentes pages en temps réel.

Pages du dashboard

Players	Tableau des joueurs connectés + graphique du nombre de joueurs dans le temps
Server	Statut en ligne/hors ligne + compteur de temps de fonctionnement (uptime)
Hardware	Graphiques en temps réel : CPU %, RAM %, Température du Raspberry Pi
Controls	Dropdown pour sélectionner un joueur, boutons Kick / Ban / Redémarrer, tableau des tickets ouverts

Boucles de mise à jour

10s	Joueurs Interroge le serveur et met à jour le tableau des joueurs et le graphique de connexions.
60s	Uptime Calcule depuis combien de temps Node-RED tourne et affiche le résultat (ex : 2h 14m 33s).
15s	Hardware Exécute une commande bash sur le Raspberry Pi pour lire le CPU, la RAM et la température.
10s	Controls Met à jour la liste déroulante des joueurs disponibles pour les actions Kick/Ban.

Configuration requise

Nœuds de configuration

Deux nœuds de configuration sont partagés par tous les flux. Ils doivent être remplis après chaque redémarrage de Node-RED.

cfg-token	Token du bot Discord "potato unpaid worker". À coller dans Configuration nodes → cfg-token.
cfg-server	Connexion RCON au serveur Minecraft. Hôte : cabma201, Port : 25575, Mot de passe : voir le serveur.

cmd-report-api — token manuel

La fonction Build Create Channel Payload contient une ligne spéciale qui nécessite le token Discord. Il faut remplacer BOT_TOKEN_HERE par le vrai token dans le code de cette fonction, directement dans Node-RED.

Permissions Discord requises

- Le bot doit avoir accès au canal Bot Tester (ID 1450828092829339855)
- Le bot doit avoir la permission Manage Channels pour créer les canaux de tickets
- Le bot doit avoir la permission Send Messages pour répondre aux commandes

Résumé — Qui fait quoi

Composant	Rôle	Communique avec
Node-RED	Cerveau central — reçoit, analyse et redistribue les messages	Discord, Minecraft, Dashboard
Discord bot	Interface utilisateur — reçoit les commandes et envoie les réponses	Node-RED
Minecraft (RCON)	Exécute les commandes reçues sur le serveur de jeu	Node-RED
Dashboard web	Affichage visuel en temps réel, contrôles manuels	Node-RED
Raspberry Pi 5	Héberge tout le système (Node-RED + serveur Minecraft)	—